



Programación y Estructura de Datos

Proyecto Final

Para este proyecto, incluya el código y capturas de pantalla que muestren el resultado de su ejecución.

Todos los códigos realizados en este proyecto deben ser escritos en Python.

El proyecto debe incluir:

- Interfaz gráfica
- Validación de entradas para evitar que el programa se rompa
- Manejo adecuado de errores
- Estructura de clases basada en Programación Orientada a Objetos

Ningún programa debe finalizar abruptamente ante entradas incorrectas.

Se debe garantizar robustez y control de excepciones.

Debe dar una breve explicación del comportamiento de las diferentes clases de su código.

El reporte debe contener el nombre completo y el código de estudiante de cada uno de los integrantes, así como la elaboración realizada en **L^AT_EX**.

Contexto

Un gimnasio requiere un sistema digital que permita controlar el acceso de sus clientes mediante el uso de un número de cliente único.

El acceso al gimnasio solo podrá realizarse mediante este número de cliente. No se permitirá el ingreso por nombre ni por ningún otro dato. Una vez que el cliente entre, el sistema deberá bloquear el uso de la tarjeta por un minuto para evitar que entre al mismo tiempo muchas veces.

El sistema deberá validar el estado de la membresía, mostrar visualmente si el acceso es permitido o denegado, registrar todos los intentos de acceso y permitir la administración de clientes y trabajadores del gimnasio mediante distintos niveles de permisos.

Objetivo General

Desarrollar un sistema de control de accesos utilizando programación orientada a objetos, base de datos relacional (SQL) e interfaz gráfica que:

- Valide el estado de las membresías.
- Registre todos los intentos de acceso.
- Controle los permisos según el rol del trabajador.
- Registre cualquier modificación realizada en la base de datos.
- Permita la exportación de información en archivos CSV.

Modalidad de Trabajo

- Equipos de 4 personas.
- Todos los integrantes deben comprender el funcionamiento completo del sistema.
- Durante la presentación cualquier integrante podrá ser cuestionado sobre cualquier parte del proyecto.

Requisitos Funcionales Obligatorios

A continuación se listan las historias de usuario de este proyecto. Úselas como una lista de verificación para determinar si su sistema cumple con los requisitos funcionales.

Gestión de administradores y empleados:

1. Como sistema, quiero tener un usuario administrador por defecto, el cual no necesite ser creado manualmente.
2. Como administrador, quiero registrar usuarios ingresando nombre, nombre de usuario, contraseña y rol (administrador o empleado), para controlar el acceso al sistema.
3. Como sistema quiero verificar que los nombres de usuario sean únicos, para evitar usuarios repetidos.
4. Como administrador, quiero actualizar los datos de los empleados, incluyendo nombre y contraseña, para mantener la seguridad del sistema.
5. Como sistema, quiero validar el usuario y contraseña del usuario (administrador o empleado) al iniciar sesión, para permitir acceso solo a credenciales correctas.
6. Como sistema, quiero restringir funcionalidades según el rol, para que solo los administradores puedan eliminar clientes.
7. Como sistema, quiero restringir funcionalidades según el rol, para que solo los administradores puedan crear otros administradores.
8. Como sistema, quiero restringir funcionalidades según el rol, para que solo los administradores crear/actualizar/eliminar empleados.
9. Como administrador, quiero visualizar la lista de usuarios (administradores o empleados) mostrando ID, nombre, nombre de usuario y rol, para supervisar al personal registrado.

Gestión de clientes:

1. Como empleado, quiero registrar un nuevo cliente ingresando su nombre e información de membresía (fecha de inicio y vencimiento), para habilitar su acceso al gimnasio.
2. Como sistema quiero generar un ID numérico único para cada cliente, para evitar clientes repetidos.
3. Como empleado, quiero actualizar los datos del cliente, incluyendo su nombre y fechas de membresía, para mantener su información actualizada.
4. Como administrador, quiero eliminar un cliente utilizando su ID, para mantener organizada la base de datos.



5. Como trabajador, quiero visualizar la lista de clientes mostrando ID, nombre, estado de la membresía (vigente o vencida) y vencimiento de membresía, para consultar su información fácilmente.
6. Como trabajador, quiero exportar la lista de clientes con todos sus atributos a un archivo CSV, para generar reportes administrativos.

Control de Acceso y Validación de Membresía:

1. Como cliente, quiero ingresar al gimnasio utilizando mi número de cliente (ID), para que el sistema valide mi acceso sin usar mi nombre ni otros datos.
2. Como sistema, quiero validar que el número de cliente (ID) exista y esté asociado a un cliente previamente registrado, para permitir el acceso únicamente a clientes válidos.
3. Como sistema, quiero verificar la fecha de inicio y fecha de vencimiento de la membresía del cliente, para determinar si está vigente, próxima a vencer (vencerá en los próximos 3 días) o vencida.
4. Como sistema, quiero negar el acceso cuando la fecha de vencimiento haya pasado, para impedir el ingreso de clientes con membresía vencida.
5. Como sistema, quiero mostrar visualmente si el acceso es permitido, permitido con advertencia (vencerá en los próximos 3 días) o denegado, para informar claramente el resultado.
6. Como sistema, quiero bloquear el número de cliente (ID) durante un minuto después de un acceso exitoso, para evitar múltiples ingresos simultáneos con el mismo número.

Requisitos No Funcionales Obligatorios

- Uso obligatorio de Programación Orientada a Objetos.
- Separación clara entre:
 - Interfaz gráfica
 - Lógica de negocio
 - Acceso a datos
- La interfaz gráfica no debe contener lógica de negocio.
- Uso de base de datos relacional.
- Integridad referencial mediante claves primarias y foráneas.
- Código organizado, modular y legible.
- Manejo correcto de fechas.
- Validación adecuada de datos.
- Control correcto de permisos según rol.



Modelo del Sistema (Clases mínimas requeridas)

Usuario

Debe incluir:

- ID
- Nombre
- Número de cliente (único)
- Estado
- Información de membresía

Membresía

Debe incluir:

- Fecha de inicio
- Fecha de vencimiento

Debe permitir determinar si está vigente o próxima a vencer.

Trabajador

Debe incluir:

- ID
- Nombre
- Usuario
- Contraseña
- Rol (admin / empleado)
- Estado de sesión

GestorUsuarios

Encargado de registrar, actualizar, eliminar (según rol) y consultar clientes. Cada modificación debe generar un registro en auditoría.

GestorTrabajadores

Encargado del registro de trabajadores (solo administrador), inicio de sesión y validación de permisos. Cada modificación debe registrarse en auditoría.

ControlAcceso

Debe validar el número de cliente, evaluar la membresía, determinar el resultado, generar el mensaje y registrar el intento.



GestorAuditoria

Debe registrar:

- Tipo de operación (INSERT, UPDATE, DELETE)
- Tabla afectada
- Trabajador responsable
- Fecha y hora
- Descripción del cambio

BaseDeDatos

Encargada de la conexión y ejecución de consultas SQL.

InterfazGráfica

Encargada únicamente de la interacción visual y la descarga de archivos CSV.

Base de Datos

Debe incluir al menos:

- usuarios
- membresias
- trabajadores
- accesos
- auditoria

Flujo General del Sistema

1. El trabajador inicia sesión.
2. El cliente ingresa su número de cliente.
3. Se valida su existencia.
4. Se evalúa la membresía.
5. Se determina el resultado.
6. Se muestra el mensaje correspondiente.
7. Se registra el intento de acceso.
8. Si hubo modificación de datos, se registra en auditoría.

Entregables

- Código fuente completo. Las clases, métodos y funciones deben estar documentadas mediante *docstrings*: <https://peps.python.org/pep-0257/>.
- Archivo(s) `.db` que contenga las bases de datos.
- Diagrama de Lenguaje Unificado de Modelado (UML).
- Documentación de usuario de su aplicación, incluir las instrucciones paso a paso (y con apoyos visuales) de las operaciones de la aplicación:
 - Iniciar la aplicación
 - Ingresar a la aplicación con el nombre de usuario y contraseña.
 - Registrar un usuario nuevo (administrador o empleado).
 - Actualizar los datos de un usuario (administrador o empleado).
 - Visualizar la lista completa de usuarios (administradores y empleados).
 - Registrar un cliente nuevo.
 - Actualizar datos de un cliente.
 - Eliminar cliente.
 - Visualizar la lista de clientes.
 - Exportar la lista de clientes.
 - Validación de membresía de usuario a través del ID.

Adicionalmente, la documentación debe contener portada e índice de contenido. Esta debe estar elaborada en L^AT_EX.

Presentación

- Duración máxima: 10 minutos.
- Ronda de preguntas: 5 minutos.
- **Todos los integrantes deben participar.**

Criterios de Evaluación

- El sistema no colapsa ante entradas inesperadas (cumple o no cumple).
- El sistema cumple los requisitos funcionales (cumple o no cumple).
- El sistema cumple los requisitos no funcionales (cumple o no cumple).
- El alumno demuestra comprensión del programa durante la demostración, no solo memorización.
- El código está bien documentado y permite el entendimiento del mismo.
- La documentación contiene las secciones requeridas. La redacción es clara y coherente. El reporte tiene una ortografía y gramática adecuadas.